DIVISÃO DE TAREFAS — 4 INTEGRANTES

Sistema de Gestão & Aluguel de Equipamentos Esportivos

Projeto Final — Desenvolvimento de Aplicações Desktop (Electron + TypeScript)

Pessoa	Área	Principais entregas
1	Arquitetura, Banco e Base Segura	Projeto base, 10 tabelas com FKs, repositórios, IPC, segurança do Electron, API própria de notificações
2	Regras de Negócio e CRUD	Entidades, CRUD principal, disponibilidade, reservas, multas, financeiro, manutenção
3	APIs Externas, Login e Dados	CEP, clima, rotas; usuários com bcrypt, perfis, validação e LGPD
4	Interface e Cadastros de Apoio	Todas as telas, CRUD de categorias/fornecedores, alerta de clima, dashboard, instalador

Requisitos obrigatórios do professor

1. Banco relacional com no mínimo 6 tabelas relacionadas (FKs) — Pessoa 1
2. CRUD completo para todas as entidades — Pessoas 2, 3 e 4
3. Consumo de API em 4 situações distintas — Pessoa 3 (3 APIs) e Pessoa 1 (API própria)
4. Segurança justificada — Pessoa 3 (login, validação, LGPD) e Pessoa 1 (Electron)
5. Código modularizado em classes (POO) — Todos (cada um tem services/classes próprias)
6. Electron com TypeScript, sem .js de lógica — Pessoas 1 e 4

Tarefas de todos os integrantes

- [ ] Cada pessoa faz commits próprios, pequenos e frequentes (pelo menos alguns por semana), com o seu nome/e-mail — o professor avalia o histórico por autor
- [ ] Adicionar o usuário willcotil como colaborador do repositório no GitHub desde o início
- [ ] Usar uma branch por tarefa e abrir Pull Requests para a main
- [ ] Revisão cruzada: cada PR é revisado por alguém de OUTRA área (1 revisa 2, 2 revisa 3, 3 revisa 4, 4 revisa 1), assim todos conhecem o sistema inteiro
- [ ] Cada pessoa escreve testes da sua própria parte
- [ ] Reunião rápida semanal para alinhar contratos de IPC/DTOs
- [ ] Usar os agentes de IA de .claude/agents/ e saber explicar cada decisão tomada com ajuda deles
- [ ] Relatório final + apresentação: cada pessoa apresenta a sua parte e todos devem conseguir explicar a arquitetura geral, as 4 APIs e as decisões de segurança

Sugestão de cronograma

Semana 1	Pessoa 1: projeto base, banco, IPC e segurança do Electron • Pessoa 2: entidades e enums • Pessoa 3: CRUD de usuários, bcrypt e login • Pessoa 4: tema, layout e roteador
Semana 2	Pessoa 1: repositórios • Pessoa 2: CRUD de clientes, equipamentos e reservas • Pessoa 3: CEP e clima • Pessoa 4: CRUD de categorias e fornecedores + telas de cadastro
Semana 3	Pessoa 1: API própria de notificações • Pessoa 2: multas, pagamentos e manutenção • Pessoa 3: rotas, validação (zod) e LGPD • Pessoa 4: reservas/devoluções e dashboard
Semana 4	Todos: testes, correções, revisão cruzada, docs/SEGURANCA.md, empacotamento e ensaio da apresentação

Pessoa 1 — Arquitetura, Banco de Dados e Base Segura

Responsável: _____

Foco: Monta a fundação que todos vão usar (projeto Electron + TypeScript, banco relacional, repositórios e IPC), deixa o Electron seguro desde o início e cria a API própria de notificações do grupo.

Requisitos cobertos: Req. 1 (6+ tabelas com FKs), Req. 3 (API 4), Req. 4 (parte Electron), Req. 6 (Electron + TS)

Agente de IA de apoio: `arquiteto-software-dados + seguranca-dados-sensiveis`

1. Configuração inicial do projeto

- [ ] Criar o projeto Electron com TypeScript (electron-vite, Electron Forge ou similar)
- [ ] Configurar tsconfig com strict: true e garantir que não exista .js de lógica
- [ ] Configurar ESLint/Prettier, scripts npm (dev, build, test) e .gitignore (.env, node_modules, banco)
- [ ] Criar a estrutura de pastas: main/, preload/, renderer/, shared/
- [ ] Escrever o README com instruções de instalação e execução

2. Modelagem do banco de dados

- [ ] Escolher o SGBD (SQLite com better-sqlite3 é o mais simples para desktop) e justificar
- [ ] Desenhar o diagrama ER (Mermaid ou draw.io) e salvar em docs/
- [ ] Tabelas: usuarios, clientes, categorias, fornecedores, equipamentos, reservas, itens_reserva, manutencoes, pagamentos, multas
- [ ] Definir chaves estrangeiras, ON DELETE/ON UPDATE e índices
- [ ] Criar scripts de migration (criação das tabelas) e seed (dados de exemplo)

3. Camada de acesso a dados (Repositórios)

- [ ] Classe de conexão com o banco (Database) com suporte a transações
- [ ] Classe base genérica de repositório (criar, buscarPorId, listar, atualizar, excluir)
- [ ] Um repositório por tabela, sempre com queries parametrizadas

4. Comunicação IPC (Main com Renderer)

- [ ] Arquivo shared/ipc-channels.ts com todos os canais no padrão dominio:acao
- [ ] DTOs tipados de entrada e saída em shared/dto/
- [ ] preload.ts expondo uma API tipada via contextBridge (window.api)
- [ ] Registro central dos handlers ipcMain.handle por domínio

5. Segurança do Electron (desde o primeiro dia)

- [ ] contextIsolation: true, sandbox: true, nodeIntegration: false, webSecurity: true
- [ ] Content-Security-Policy restritiva no HTML (default-src 'self')
- [ ] Bloquear navegação externa e window.open (will-navigate, setWindowOpenHandler)
- [ ] Validar o remetente (event.senderFrame) nos handlers IPC sensíveis
- [ ] Escrever a seção "Electron" do docs/SEGURANCA.md

6. Integração 4 — API própria de notificações

- [ ] Pequena API do grupo em TypeScript (Express) com rota de lembrete de devolução
- [ ] Envio de lembrete/aviso de multa por e-mail (Mailtrap/Resend) ou WhatsApp simulado
- [ ] Classe NotificacaoClient no Main Process + canal IPC notificacoes:enviarLembrete

7. Documentação técnica

- [ ] Documento de arquitetura em camadas (diagrama + explicação de cada camada)
- [ ] Dicionário de dados (tabela, campos, tipos e relacionamentos)

Pessoa 2 — Regras de Negócio e Fluxo de Aluguel

Responsável: _____

Foco: Implementa o coração do sistema de aluguel em classes TypeScript: entidades, services, CRUD das entidades principais e as regras de reserva, multa, pagamento e manutenção.

Requisitos cobertos: Req. 2 (CRUD das entidades principais), Req. 5 (lógica em classes)

Agente de IA de apoio: `regras-negocio-dominio`

1. Entidades de domínio (classes)

- ☐ Classes: Usuario, Cliente, Categoria, Fornecedor, Equipamento, Reserva, ItemReserva, Manutencao, Pagamento, Multa
- ☐ Enums: Modalidade (VENDA/LOCACAO), EstadoConservacao (NOVO/MUITO_BOM/MEDIO/REGULAR), StatusEquipamento, StatusReserva, TipoFornecedor
- ☐ Classes de erro de domínio (RegraNegocioError, EquipamentoIndisponivelError)

2. CRUD (services + handlers IPC)

- ☐ Clientes e Equipamentos
- ☐ Reservas e Itens de Reserva
- ☐ Manutenções, Pagamentos e Multas
- ☐ (Categorias e Fornecedores ficam com a Pessoa 4; Usuários com a Pessoa 3)

3. Inventário e disponibilidade

- ☐ Somente itens de LOCACAO podem ser reservados
- ☐ Verificação de disponibilidade por período (sem sobreposição de reservas e sem manutenção aberta)
- ☐ Atualização automática do status do equipamento (Disponível, Reservado, Em Manutenção, Indisponível)

4. Reservas, devoluções e multas

- ☐ Criar reserva com vários itens dentro de uma transação
- ☐ Ciclo de status: PENDENTE » ATIVA » DEVOLVIDA / CANCELADA (bloquear transições inválidas)
- ☐ Registrar devolução com o novo estado de conservação
- ☐ Cálculo automático de multa por atraso (diária fixa ou percentual — padrão Strategy)
- ☐ Listagem de devoluções pendentes e atrasadas

5. Financeiro e histórico

- ☐ Registro de pagamento do aluguel, caução (retida/devolvida) e quitação de multas
- ☐ Histórico do cliente: reservas ativas, pendências, pagamentos e multas

6. Manutenção

- ☐ Enviar equipamento para manutenção e registrar retorno (data, custo, novo estado)
- ☐ Validar vínculos: equipamento com fornecedor de COMPRA; manutenção com prestador de MANUTENCAO

7. Testes

- ☐ Testes unitários (Vitest/Jest) para disponibilidade, multas e transições de status

Pessoa 3 — APIs Externas, Login e Proteção de Dados

Responsável: _____

Foco: Implementa as 3 integrações com APIs públicas, o login com controle de acesso e a validação/proteção dos dados que entram no sistema.

Requisitos cobertos: Req. 3 (APIs 1, 2 e 3), Req. 4 (autenticação, validação, LGPD)

Agente de IA de apoio: `especialista-apis-integracoes + seguranca-dados-sensiveis`

1. Integração 1 — CEP (ViaCEP)

- ☐ Classe ViaCepClient no Main Process + canal IPC integracoes:buscarCep
- ☐ Preenchimento automático de rua, bairro, cidade e UF no cadastro de cliente
- ☐ Tratar CEP inválido, CEP inexistente e falta de internet (permitir digitação manual)

2. Integração 2 — Previsão do Tempo

- ☐ Classe ClimaClient (OpenWeatherMap ou Open-Meteo)
- ☐ Consultar o clima da data de retirada na confirmação da reserva
- ☐ Gerar alerta para categorias sensíveis ao clima (caiaque, bicicleta, prancha)

3. Integração 3 — Rotas / Mapas

- ☐ Classe RotasClient (OpenRouteService ou OSRM + Nominatim)
- ☐ Calcular distância e tempo até o cliente com devolução atrasada (busca presencial)

4. Boas práticas das integrações

- ☐ Timeout com AbortController, cache em memória e mensagens de erro amigáveis
- ☐ Chaves de API em .env (fora do Git) + .env.example versionado
- ☐ Testes com respostas mockadas

5. Autenticação e controle de acesso

- ☐ CRUD de Usuários com senhas em bcrypt (nunca em texto puro)
- ☐ Fluxo de login no Main Process, limite de tentativas e mensagem de erro genérica
- ☐ Perfis ADMIN e FUNCIONARIO com permissões verificadas nos handlers IPC

6. Validação e dados sensíveis

- ☐ Validação de todos os dados recebidos via IPC (zod): CPF, CEP, e-mail, datas, valores
- ☐ Proteção de dados sensíveis (LGPD): CPF mascarado e logs sem dados pessoais

7. Documento de segurança

- ☐ Organizar o docs/SEGURANCA.md justificando cada decisão (exigido pelo professor)

Pessoa 4 — Interface (UI/UX) e Cadastros de Apoio

Responsável: _____

Foco: Constrói as telas da aplicação desktop, conectando-as ao Main Process via window.api, e faz de ponta a ponta (lógica + tela) os cadastros de Categorias e Fornecedores.

Requisitos cobertos: Req. 2 (telas de CRUD + CRUD de Categorias/Fornecedores), Req. 5, Req. 6 (Renderer em TS)

Agente de IA de apoio: `interface-ui-electron + regras-negocio-dominio`

1. Base visual

- [ ] Escolher a abordagem (TypeScript puro com classes ou React/Vue com Vite + TS)
- [ ] Tema em CSS (variáveis de cores e espaçamento), layout com menu lateral
- [ ] Roteador simples entre telas e componentes reutilizáveis (tabela, formulário, modal, toast)
- [ ] Estados de carregando, vazio e erro em todas as telas

2. CRUD completo de Categorias e Fornecedores (lógica + tela)

- [ ] CategoriaService e FornecedorService (classes) + handlers IPC
- [ ] Campo sensivel_clima na categoria (usado no alerta de previsão do tempo)
- [ ] Fornecedores com abas COMPRA / MANUTENÇÃO e validação de CNPJ
- [ ] Testes unitários dos dois services

3. Telas de acesso e painel

- [ ] Login
- [ ] Dashboard: reservas ativas, devoluções atrasadas, equipamentos em manutenção e multas em aberto

4. Telas de cadastro

- [ ] Inventário de equipamentos (filtros por categoria, modalidade, status e conservação)
- [ ] Clientes (CEP com preenchimento automático) e histórico do cliente
- [ ] Usuários (somente ADMIN)

5. Fluxo de reservas e devoluções

- [ ] Assistente de reserva: cliente » período » equipamentos disponíveis » confirmação
- [ ] Card de previsão do tempo e alerta de clima na confirmação
- [ ] Painel de devoluções (ativas, atrasadas, devolvidas), com multa calculada e botão de rota
- [ ] Telas de pagamentos/caução e de manutenções (envio e retorno)

6. Qualidade da interface

- [ ] Confirmação antes de excluir, feedback após salvar e validação visual dos formulários
- [ ] Acessibilidade: labels, foco visível, contraste e navegação por teclado
- [ ] Respeitar a CSP (sem scripts inline e sem innerHTML com dados do usuário)

7. Empacotamento

- [ ] Gerar o instalador/executável (electron-builder ou Electron Forge)
- [ ] Capturas de tela para o relatório e a apresentação